

Packages: NumPy, pandas, and matplotlib

Machine Learning in Finance · Session 3

Last session, you wrote this

Volatility for one stock, built by hand out of loops:

Python Code

```
def volatility(prices):  
    returns = []  
    for i in range(1, len(prices)):  
        returns.append((prices[i] - prices[i - 1]) / prices[i - 1])  
    mean = sum(returns) / len(returns)  
    squared = 0  
    for r in returns:  
        squared += (r - mean) ** 2  
    return (squared / len(returns)) ** 0.5  
  
volatility([183.56, 182.19, 179.87, 179.15, 183.48])
```

0.014300824856182556

Now do the whole risk report

The real data is one file: **eleven stocks, ten years, 27,676 rows.**

```
date,ticker,close,volume  
2015-01-02,AAPL,24.1926,212818400  
2015-01-05,AAPL,23.5111,257142000  
...  
2024-12-31,XOM,102.3799,11302800
```

With lists and loops you would first have to split every line at the commas, turn the text into numbers, and sort the rows into eleven separate lists. All before computing anything.

The same answer, in three lines

```
prices = pd.read_csv("data/prices.csv", parse_dates=["date"])  
daily = prices.pivot(index="date", columns="ticker", values="close").pct_change()  
daily.std().sort_values(ascending=False)
```

```
ticker  
NVDA    0.0306  
AAPL    0.0179  
DIS     0.0175  
XOM     0.0175  
JPM     0.0172  
MSFT    0.0171  
WMT     0.0133  
PG      0.0116  
JNJ     0.0114  
KO      0.0112  
SPY     0.0111
```

None of that is meant to make sense yet. It is where we are going.

That is what a package is

A **package** is code somebody else wrote, that you can use by name. Three of them do most of the work in data analysis:

NumPy

numbers, vectors and matrices, and the maths on them

pandas

tables with named columns, like a spreadsheet you can program

matplotlib

figures, so you can see what you found

There are thousands of others, and you will meet more of them as the course goes on, including **scikit-learn**, the one that fits models.

And they live on your machine

Packages are not part of Python. Somebody has to install them, and in Colab somebody already did.

At the exam you sit with **your own laptop, VS Code, and no internet**. So today starts there: your machine, your packages, and how to look things up when you cannot search for them.

Today

By the end of this session you can:

- work in **VS Code** on your own machine, and find documentation offline
- use **NumPy** for arrays, and for the matrix algebra behind every model
- use **pandas** to load, filter and summarise a real table of prices
- use **matplotlib** to turn a result into a figure somebody else can read

And the case reaches all eleven stocks at once.

1 • Your own machine

PART 1

VS Code, packages, and finding things out without the internet.

Why leave Colab

Colab still works, and the homework stays there. But:

The exam

three hours, your laptop, VS Code, offline. It should not be the first time.

It is yours

your files, your packages, nothing to upload, no session that expires

The editor helps you

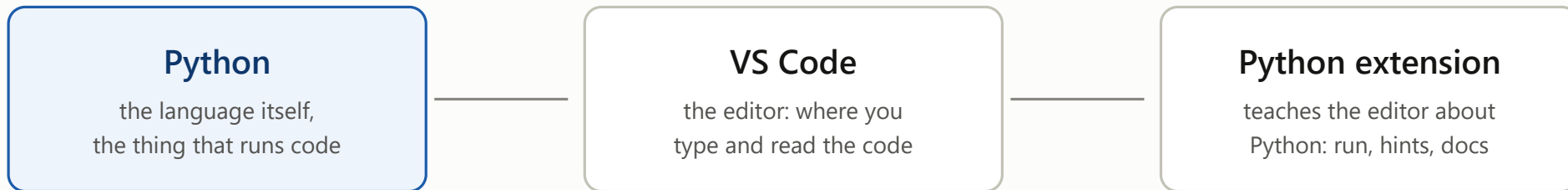
it completes names and shows documentation as you type

No internet needed

once it is installed, it works on a train

Three things, not one

The commonest confusion is thinking VS Code *is* Python. It is not:



The setup guide installs all three.

You still work in notebooks

VS Code opens `.ipynb` files: the same cells, the same Run buttons, the same output underneath as in Colab.

Nothing about how you write code changes. Only where it runs. The exam is written in a notebook too.

The first time you open one, VS Code asks which Python to use. Pick the one you installed, and it remembers.

Installing a package

Packages come from a public archive called PyPI. You install one **once**, and then it is on your machine for good:

```
%pip install pandas
```

Run that in a notebook cell, like any other code. The `%` marks it as an instruction to the notebook rather than to Python. Your setup guide already installed the three we need, so you should not have to type it at all today.

Installing and importing are different. You **install** a package once, ever. You **import** it in every notebook that uses it.

The import line

import brings a package in. **as** gives it a short name:

Python Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.mean([1, 2, 3, 4])
```

```
np.float64(2.5)
```

np, **pd** and **plt** are conventions, not rules. Everybody uses them and every tutorial assumes them, so use them too.

When it cannot find it

```
import pandas as pd
```

```
ModuleNotFoundError: No module named 'pandas'
```

Python is telling you the truth: it looked, and it is not there. Install it, then run the import again.

Same habit as Session 1. The message names the thing that is missing.

Your data sits next to your notebook

```
mlfin/  
├─ analysis.ipynb  
└─ data/  
    └─ prices.csv
```

```
prices = pd.read_csv("data/prices.csv")
```

No upload step any more. If Python says `FileNotFoundError`, the file is somewhere other than where you said. Open the **folder** in VS Code, not just the file, and the paths work.

Looking things up

pandas has hundreds of functions, each with its own arguments. Nobody memorises them, and professionals look things up all day. At the exam you have no internet.

Two things cover you. Before the exam I will give you a **cheatsheet** with every function you are likely to need. And the documentation for each package is already **inside the package**, on your machine.

The cheatsheet is on the course page, under [Cheatsheet](#): every function we have used, with a worked example of each.

VS Code shows it as you type

Rest the mouse on any name:

```
1 daily.std().sort_values(ascending=False)
```

```
sort_values(ascending: bool = True, ...) → Series
```

Sort by the values.

ascending : bool, default True

If True, sort values in ascending order, otherwise descending.

That card is not from the internet. It is the text that ships inside pandas, the same text `help()` prints.

The official documentation

For learning something new, rather than recalling a detail, the websites are much better. They have a written guide, worked examples, and pictures:

- docs.python.org for the language itself
- numpy.org/doc for arrays and linear algebra
- pandas.pydata.org/docs for tables
- matplotlib.org for figures, including a **gallery** of example charts

Use these at home while you work. They are also downloadable as a single file, if you want them offline. All of them, and more, are linked from [Resources](#) on the course page.

2 • NumPy

PART 2

Arrays: doing arithmetic to a whole list at once.

Somebody already wrote your function

Your `volatility` is a standard deviation, and standard deviations have been written many times before:

Python Code

```
import numpy as np

returns = np.array([0.012, -0.004, 0.008, -0.011, 0.006])
np.std(returns)
```

```
np.float64(0.00844748483277715)
```

Writing it yourself was how you learned what it means. From now on you use the version that is already written, tested and fast.

A list, and an array

An **array** is NumPy's version of a list: same idea, all numbers, and the whole thing moves at once.

Python Code

```
close = np.array([183.56, 182.19, 179.87, 179.15, 183.48])  
close
```

```
array([183.56, 182.19, 179.87, 179.15, 183.48])
```

You build it from a list, and you index and slice it exactly like a list. Everything from Session 1 still works.

Arithmetic on the whole array

Here is the difference. Multiply an array, and every element is multiplied:

Python Code

```
close = np.array([183.56, 182.19, 179.87, 179.15, 183.48])  
close * 2
```

```
array([367.12, 364.38, 359.74, 358.3 , 366.96])
```

No loop, no empty list, no `.append`. In Session 2 this was three lines.

Two arrays at once

Python Code

```
a = np.array([100.0, 200.0, 300.0])  
b = np.array([1.0, 2.0, 3.0])  
a - b
```

```
array([ 99., 198., 297.])
```

Position by position: **100-1**, **200-2**, **300-3**.

Every daily return, in one line

Slicing from Session 1, plus that arithmetic:

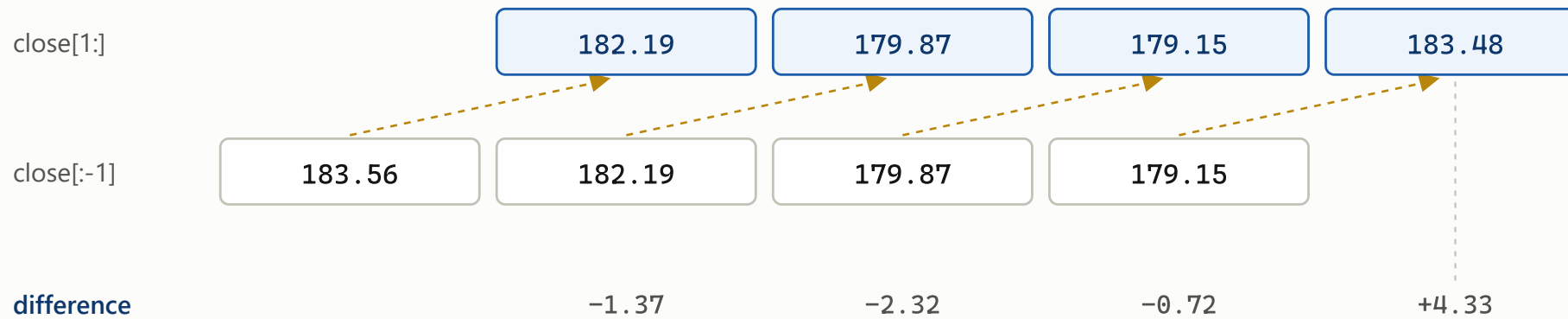
Python Code

```
close = np.array([183.56, 182.19, 179.87, 179.15, 183.48])
returns = (close[1:] - close[:-1]) / close[:-1]
returns
```

```
array([-0.0074635 , -0.01273396, -0.00400289,  0.02416969])
```

`close[1:]` is every day from the second. `close[:-1]` is every day except the last. Subtracting them lines each day up against the day before.

The same idea, drawn



The arrows show the pairing: each day is lined up against the day before it, which is exactly what the subtraction does.

Summarising an array

The things you built by hand in Session 2 are already there:

Python Code

```
returns = np.array([0.012, -0.004, 0.008, -0.011, 0.006])  
print("mean :", returns.mean())  
print("std  :", returns.std())  
print("min  :", returns.min())  
print("max  :", returns.max())
```

```
mean : 0.0022  
std  : 0.00844748483277715  
min  : -0.011  
max  : 0.012
```

`returns.std()` is your whole **volatility** function.

A question asked of every element

Compare an array to a number, and you get an answer for each element:

Python Code

```
returns = np.array([0.012, -0.004, 0.008, -0.011, 0.006])  
returns > 0
```

```
array([ True, False,  True, False,  True])
```

This is the Session 2 **if**, applied to everything at once instead of one item at a time inside a loop.

Counting with that answer

Python Code

```
returns = np.array([0.012, -0.004, 0.008, -0.011, 0.006])  
(returns > 0).sum()
```

```
np.int32(3)
```

Adding up **True** and **False** counts the **Trues**, because **True** counts as one. That is the up-day counter from Session 2, in one line.

Annualising, while we are here

Daily volatility is a small number. Finance quotes it per year, by multiplying by the square root of the number of trading days:

Python Code

```
daily_vol = 0.0143  
daily_vol * np.sqrt(252)
```

```
np.float64(0.2270054624893419)
```

So Apple's 1.4% a day is about 23% a year. You will see `np.sqrt(252)` a lot.

Your turn 1

Compute the daily returns from these prices.

Python Code

```
close = np.array([100.0, 102.0, 101.0, 104.0, 103.5])
```

```
returns = ...  
returns
```

Your turn 1: answer

```
close = np.array([100.0, 102.0, 101.0, 104.0, 103.5])  
  
returns = (close[1:] - close[:-1]) / close[:-1]  
returns
```

Four returns from five prices: +2.0%, -1.0%, +3.0%, -0.5%.

From lists of numbers to tables of them

So far, one row of numbers. NumPy's other subject is the **matrix**: rows and columns of them.

You have done all of this maths in econometrics. Nothing on the next few slides is a new idea. What is new is that each one takes a single line.

The prerequisites

Vector

A row or column of numbers. One stock's returns, or a set of portfolio weights.

Transpose

Flip rows and columns. An $n \times p$ becomes a $p \times n$.

Matrix product

Only fits if the first one's columns match the second one's rows.

Matrix

A rectangle of numbers, n rows by p columns. One row per observation.

Dot product

Multiply pairwise, then add up. Weights times returns is a portfolio return.

Inverse

The matrix that undoes another one. Exists only if no column is redundant.

A matrix is a list of rows

Python Code

```
X = np.array([[1.0, 0.012],  
              [1.0, -0.004],  
              [1.0, 0.008]])  
  
print(X)  
print("shape:", X.shape)
```

```
[[ 1.    0.012]  
 [ 1.   -0.004]  
 [ 1.    0.008]]  
shape: (3, 2)
```

Three rows, two columns. **shape** always tells you which is which.

Reaching into it

Python Code

```
print("one element :", X[0, 1])  
print("first row   :", X[0, :])  
print("second column:", X[:, 1])
```

```
one element   : 0.012  
first row     : [1.      0.012]  
second column: [ 0.012 -0.004  0.008]
```

[row, column], and : means “all of them”. The same slicing as a list, once per direction.

Transpose

.T flips it:

Python Code

```
print("X   is", X.shape)
print("X.T is", X.T.shape)
X.T
```

```
X   is (3, 2)
X.T is (2, 3)
```

```
array([[ 1.    ,  1.    ,  1.    ],
       [ 0.012, -0.004,  0.008]])
```

Multiplying: the dot product

Multiply pairwise and add up. In finance that is a **portfolio return**: weights times returns.

Python Code

```
weights = np.array([0.5, 0.3, 0.2])
returns = np.array([0.012, -0.004, 0.008])

weights @ returns
```

```
np.float64(0.0064)
```

$0.5 \times 0.012 + 0.3 \times (-0.004) + 0.2 \times 0.008$. The @ symbol is matrix multiplication, and for two vectors that is the dot product.

The same @, on a matrix

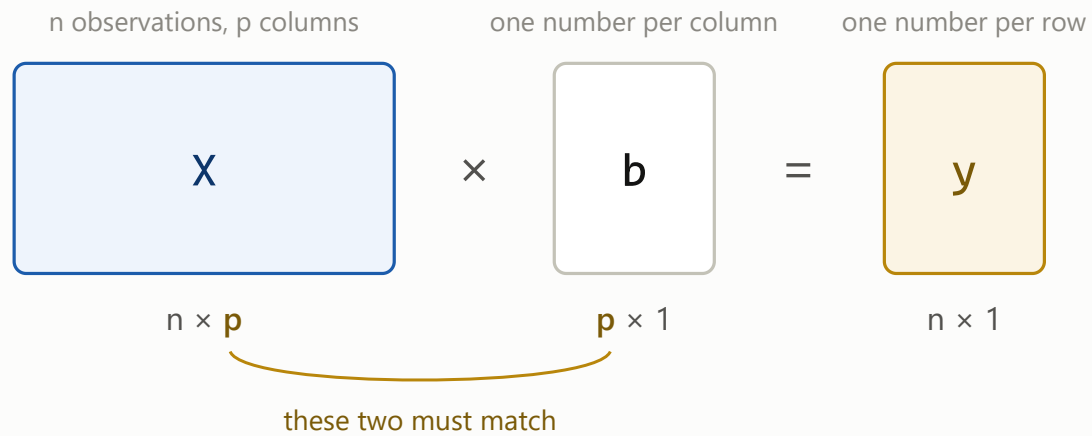
Python Code

```
X = np.array([[1.0, 0.012],  
              [1.0, -0.004],  
              [1.0, 0.008]])  
b = np.array([0.001, 1.2])  
  
X @ b
```

```
array([ 0.0154, -0.0038,  0.0106])
```

Three rows in, three numbers out: one per row. That is a regression's fitted values, for the whole sample in one go.

Shapes have to fit



Nearly every NumPy error you will ever see is this rule.

The inverse

`np.linalg.inv` gives the matrix that undoes another one:

Python Code

```
A = np.array([[2.0, 1.0],  
              [1.0, 3.0]])  
  
np.linalg.inv(A) @ A
```

```
array([[ 1.00000000e+00, -1.11022302e-16],  
       [ 0.00000000e+00,  1.00000000e+00]])
```

The result is the **identity**: ones down the diagonal, zeros elsewhere. The off-diagonal one prints as something like `-1.11e-16`, which is scientific notation for a decimal point followed by fifteen zeros and a 111. Computers store decimals approximately, so a value that should be exactly zero often comes out this tiny instead. Read it as zero.

When there is no inverse

```
B = np.array([[1.0, 2.0],  
              [2.0, 4.0]])
```

```
np.linalg.inv(B)
```

```
LinAlgError: Singular matrix
```

The second column is just twice the first, so it adds nothing.

You have met this before under another name: perfect multicollinearity, the regression that will not run.

OLS, in one line

Your econometrics course wrote the estimator like this:

$$\hat{\beta} = (X'X)^{-1}X'y$$

```
beta = np.linalg.solve(X.T @ X, X.T @ y)
```

Read it against the formula: **X.T** is X' , **@** is multiplication, and **solve** does the inverting and multiplying in one safer step.

A real regression: market beta

Apple's daily return against the market's. The slope is the **beta** you know from the CAPM:

Python Code

```
px = pd.read_csv("data/prices.csv", parse_dates=["date"])
rets = px.pivot(index="date", columns="ticker", values="close").pct_change().dropna()

y = rets["AAPL"].values
x = rets["SPY"].values
X = np.column_stack([np.ones(len(x)), x])

alpha, beta = np.linalg.solve(X.T @ X, X.T @ y)
print("alpha:", round(alpha, 6), " beta:", round(beta, 3))
```

```
alpha: 0.000426  beta: 1.207
```

Change the ticker and run it again: Nvidia gives 1.75, JPMorgan 1.12, Coca-Cola 0.60. Above one moves more than the market, below one moves less.

Why arrays are worth it

Squaring a million numbers, first with a loop and then with an array:

```
import time
xs, arr = list(range(1_000_000)), np.arange(1_000_000)

t0 = time.perf_counter()
total = 0
for v in xs:
    total += v * v
loop = time.perf_counter() - t0

t0 = time.perf_counter()
total = (arr * arr).sum()
vec = time.perf_counter() - t0

print(f"loop: {loop:.3f} s      numpy: {vec:.4f} s      {loop / vec:.0f} times faster")
```

```
loop: 0.079 s      numpy: 0.0021 s      37 times faster
```

NumPy hands the whole array to compiled code, instead of stepping through it one Python statement at a time.

3 • pandas

PART 3

Tables with names on the columns.

What pandas adds

NumPy gives you numbers in a grid. pandas puts **labels** on that grid:

Series

one column of values, each with a label. A list and a dictionary at once.

DataFrame

several Series side by side, sharing one index. A spreadsheet you can program.

A Series

Python Code

```
import pandas as pd

vol = pd.Series({"AAPL": 0.0143, "KO": 0.0081, "NVDA": 0.0331})
vol
```

```
AAPL    0.0143
KO       0.0081
NVDA     0.0331
dtype: float64
```

Built straight from a Session 2 dictionary.

A Series does both jobs

Python Code

```
print(vol["NVDA"])    # by name, like a dictionary
print(vol.mean())     # summarises itself, like an array
vol * np.sqrt(252)    # and does maths all at once
```

```
0.0331
0.0185
```

```
AAPL    0.227005
KO       0.128584
NVDA     0.525446
dtype: float64
```

A DataFrame

Python Code

```
df = pd.DataFrame({  
    "ticker": ["AAPL", "KO", "NVDA"],  
    "vol": [0.0143, 0.0081, 0.0331],  
    "beta": [1.21, 0.60, 1.75],  
})  
df
```

	ticker	vol	beta
0	AAPL	0.0143	1.21
1	KO	0.0081	0.60
2	NVDA	0.0331	1.75

Columns have names, and different columns can hold different types.

Reading a real file

In Sessions 1 and 2 the data arrived in a cell somebody wrote for you. Not any more:

Python Code

```
prices = pd.read_csv("data/prices.csv", parse_dates=["date"])
prices.head()
```

	date	ticker	close	volume
0	2015-01-02	AAPL	24.1926	212818400
1	2015-01-05	AAPL	23.5111	257142000
2	2015-01-06	AAPL	23.5133	263188400
3	2015-01-07	AAPL	23.8430	160423600
4	2015-01-08	AAPL	24.7591	237458000

`.head()` shows the first five rows. `parse_dates` turns that column from text into real dates, which is what lets you sort and slice by time later.

Look before you compute

Python Code

```
print("shape :", prices.shape)
print()
print(prices.dtypes)
```

```
shape : (27676, 4)
```

```
date      datetime64[ns]
ticker      object
close      float64
volume     int64
dtype: object
```

One column

Python Code

```
prices["close"].head()
```

```
0    24.1926
```

```
1    23.5111
```

```
2    23.5133
```

```
3    23.8430
```

```
4    24.7591
```

```
Name: close, dtype: float64
```

A name in square brackets gives back a **Series**: that column, with its row labels kept.

Several columns

A **list** of names inside the brackets gives a smaller **DataFrame**:

Python Code

```
prices[["date", "close"]].head()
```

	date	close
0	2015-01-02	24.1926
1	2015-01-05	23.5111
2	2015-01-06	23.5133
3	2015-01-07	23.8430
4	2015-01-08	24.7591

One set of brackets for the indexing, one for the list. That is why there are two of them.

Keeping only some rows

A condition gives **True** or **False** for every row, just as in NumPy. Put it in brackets to keep the **True** ones:

Python Code

```
aapl = prices[prices["ticker"] == "AAPL"]  
aapl.head()
```

	date	ticker	close	volume
0	2015-01-02	AAPL	24.1926	212818400
1	2015-01-05	AAPL	23.5111	257142000
2	2015-01-06	AAPL	23.5133	263188400
3	2015-01-07	AAPL	23.8430	160423600
4	2015-01-08	AAPL	24.7591	237458000

27,676 rows tested at once.

Adding a column

Assign to a name that does not exist yet, and it appears:

Python Code

```
aapl = prices[prices["ticker"] == "AAPL"].copy()
aapl["ret"] = aapl["close"].pct_change()
aapl[["date", "close", "ret"]].head()
```

	date	close	ret
0	2015-01-02	24.1926	NaN
1	2015-01-05	23.5111	-0.028170
2	2015-01-06	23.5133	0.000094
3	2015-01-07	23.8430	0.014022
4	2015-01-08	24.7591	0.038422

`.pct_change()` is the daily return: the shift-and-divide you have now written by hand twice.

Why the `.copy()`

Leave it out, and pandas complains:

```
aapl = prices[prices["ticker"] == "AAPL"]      # no .copy()
aapl["ret"] = aapl["close"].pct_change()
```

```
SettingWithCopyWarning: A value is trying to be set on a copy of a
slice from a DataFrame.
```

`aapl` might be a window onto `prices` rather than a table of its own, so pandas cannot tell whether you also meant to change the original. `.copy()` says “give me my own table”, and the warning goes away.

The first one is missing

The first day has no day before it, so its return is **NaN**, “not a number”:

Python Code

```
aapl["ret"].head(3)
```

```
0      NaN
1  -0.028170
2   0.000094
Name: ret, dtype: float64
```

NaN means “no value here”. It is not zero and it is not an error, and most summary methods skip it for you.

Apple's volatility

Python Code

```
print("daily      :", aapl["ret"].std().round(5))  
print("annualised:", (aapl["ret"].std() * np.sqrt(252)).round(4))
```

```
daily      : 0.01793  
annualised: 0.2846
```

Ten years of Apple: about 1.8% on a typical day, 28% a year. Your Session 2 case found 1.4% for 2024 alone, which was a calmer year.

Sorting

Python Code

```
aapl.sort_values("ret").head(3)[["date", "close", "ret"]]
```

	date	close	ret
1308	2020-03-16	58.4703	-0.128647
1007	2019-01-03	33.7370	-0.099607
1306	2020-03-12	59.9236	-0.098755

Apple's three worst days in ten years, dates and all.

Your turn 2

Pick out Coca-Cola, then read off its daily volatility.

Python Code

```
ko = prices[prices["ticker"] == ...].copy()
ko["ret"] = ...
ko["ret"].std()
```

Your turn 2: answer

```
ko = prices[prices["ticker"] == "KO"].copy()
ko["ret"] = ko["close"].pct_change()
ko["ret"].std()
```

About 0.0112, so 1.1% a day against Apple's 1.8%. The comparison you began in Session 1 with two hand-picked prices, now over ten years.

All eleven at once

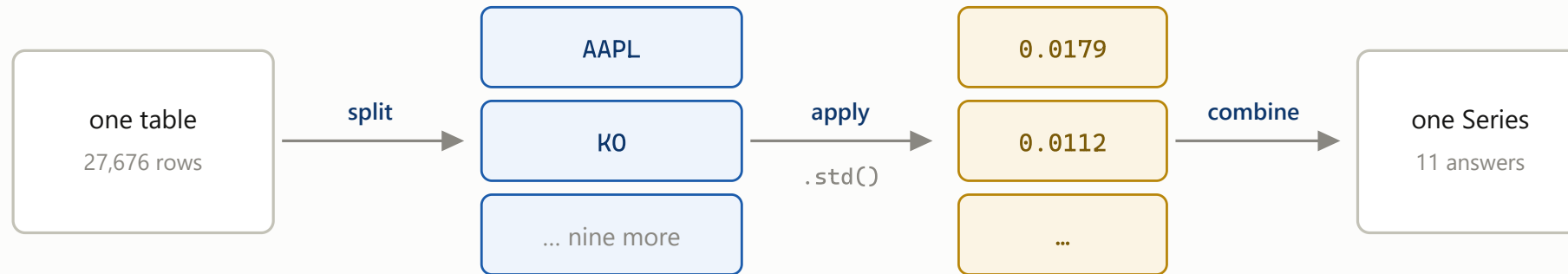
Doing that eleven times would be eleven copies of the same code. Instead, **group** by ticker and let pandas handle each group:

Python Code

```
prices["ret"] = prices.groupby("ticker")["close"].pct_change()  
prices.groupby("ticker")["ret"].std().sort_values(ascending=False)
```

ticker	
NVDA	0.030620
AAPL	0.017929
DIS	0.017537
XOM	0.017504
JPM	0.017241
MSFT	0.017096

groupby, in words



Split, apply, combine. You will see this shape everywhere.

Several summaries at once

Python Code

```
prices.groupby("ticker")["ret"].agg(["mean", "std", "min", "max"]).round(4)
```

	mean	std	min	max
ticker				
AAPL	0.0011	0.0179	-0.1286	0.1198
DIS	0.0003	0.0175	-0.1316	0.1441
JNJ	0.0003	0.0114	-0.1004	0.0800
JPM	0.0008	0.0172	-0.1496	0.1801
KO	0.0003	0.0112	-0.0967	0.0648
MSFT	0.0011	0.0171	-0.1474	0.1422
NVDA	0.0027	0.0306	-0.1876	0.2980
PG	0.0004	0.0116	-0.0874	0.1201
SPY	0.0005	0.0111	-0.1094	0.0906
WMT	0.0006	0.0133	-0.1138	0.1171
XOM	0.0004	0.0175	-0.1222	0.1269

One column per stock

`pivot` reshapes the table: dates down the side, tickers across the top.

Python Code

```
wide = prices.pivot(index="date", columns="ticker", values="close")
wide.tail(3)
```

ticker	AAPL	DIS	JNJ	JPM	KO	MSFT	NVDA	PG	SPY	WMT	XOM
date											
2024-12-27	253.9674	109.4909	139.1970	232.8639	59.8596	425.4825	136.8057	162.7761	584.9950	90.4324	101.3425
2024-12-30	250.5989	108.7547	137.5560	231.0776	59.4570	419.8493	137.2849	160.4333	578.3193	89.3570	100.6572
2024-12-31	248.8302	109.2946	138.7844	231.4541	59.6775	416.5584	134.0897	160.9710	576.2153	89.1400	102.3799

The tall shape is better for storing and grouping. This wide one is better for comparing stocks and for plotting, which is where we go next.

And it does maths column by column

Python Code

```
wide.pct_change().std().sort_values(ascending=False).round(4)
```

ticker	
NVDA	0.0306
AAPL	0.0179
DIS	0.0175
XOM	0.0175
JPM	0.0172
MSFT	0.0171
WMT	0.0133
PG	0.0116

Picking out a date

Because the index is dates, you can slice it with dates:

Python Code

```
wide.loc["2024-01-02":"2024-01-08", ["AAPL", "KO"]]
```

ticker date	AAPL	KO
2024-01-02	183.5622	55.6394
2024-01-03	182.1878	55.7696
2024-01-04	179.8739	55.5836
2024-01-05	179.1521	55.4999
2024-01-08	183.4831	55.9091

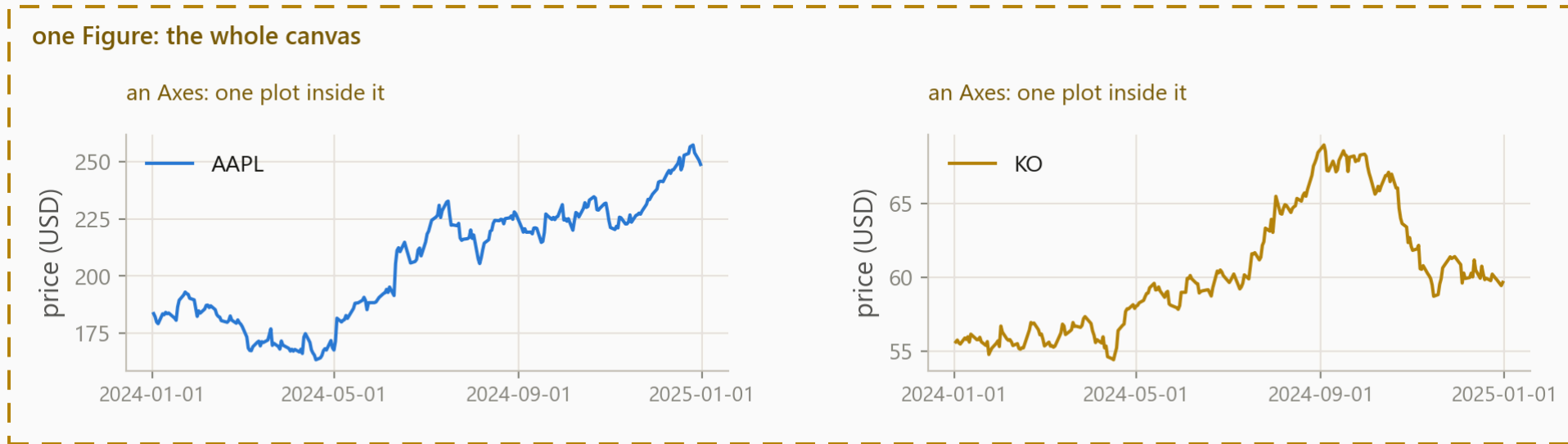
`.loc` works with the **labels** you can see. There is also `.iloc` for positions, if you ever want “the last row” rather than a particular date.

4 • matplotlib

PART 4

Drawing what you found, so somebody else can see it.

A figure is made of two things



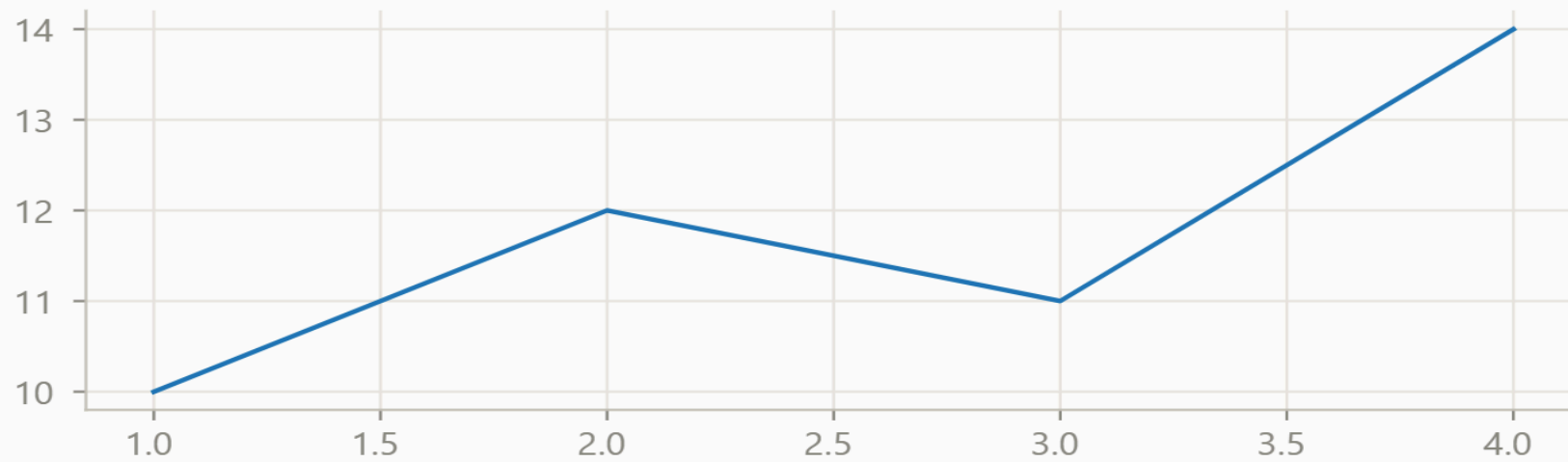
A **Figure** is the canvas; an **Axes** is one plot on it. Everything you draw, you draw on an Axes.

Every plot starts the same way

Python Code

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7.5, 2.4))
ax.plot([1, 2, 3, 4], [10, 12, 11, 14])
plt.show()
```



Title, label, colour

Python Code

```
aapl24 = wide.loc["2024", "AAPL"]

fig, ax = plt.subplots(figsize=(8, 2.0))
ax.plot(aapl24.index, aapl24.values, color="#1c5cab", linewidth=1.6)
ax.set_title("Apple (AAPL) closing price, 2024", loc="left")
ax.set_ylabel("price (USD)")
plt.show()
```



Two lines, and a legend

Python Code

```
p24 = wide.loc["2024"]  
fig, ax = plt.subplots(figsize=(8, 2.1))  
ax.plot(p24.index, p24["AAPL"], label="AAPL")  
ax.plot(p24.index, p24["KO"], label="KO")  
ax.set_ylabel("price (USD)")  
ax.legend()  
plt.show()
```

Your turn 3

Give this plot a title and a y-axis label.

Python Code

```
nvda = wide.loc["2024", "NVDA"]

fig, ax = plt.subplots(figsize=(8, 2.2))
ax.plot(nvda.index, nvda.values)
...
plt.show()
```

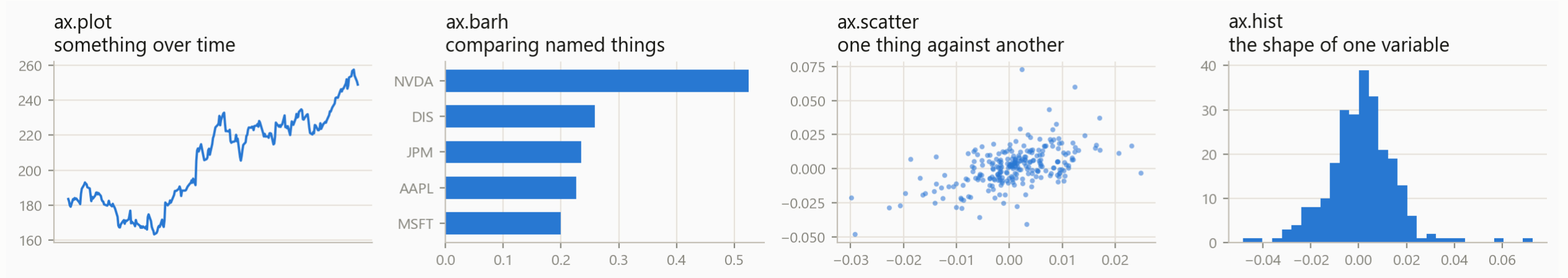
Your turn 3: answer

```
nvda = wide.loc["2024", "NVDA"]

fig, ax = plt.subplots(figsize=(8, 2.2))
ax.plot(nvda.index, nvda.values)
ax.set_title("Nvidia (NVDA) closing price, 2024", loc="left")
ax.set_ylabel("price (USD)")
plt.show()
```

A tripling over the year, and a reader who can tell what they are looking at.

Four kinds of plot



Four calls, and the question decides which one you want. Everything else on a plot is decoration you add afterwards.

Bars, for comparing stocks

Python Code

```
vol = wide.pct_change().std().sort_values()

fig, ax = plt.subplots(figsize=(8, 2.4))
ax.barh(vol.index, vol.values, color="#2a78d6")
ax.set_title("Daily volatility, 2015 to 2024", loc="left")
plt.show()
```

A histogram, for a distribution

Python Code

```
r = wide["AAPL"].pct_change().dropna()

fig, ax = plt.subplots(figsize=(8, 2.2))
ax.hist(r, bins=50, color="#2a78d6")
ax.set_title("Apple daily returns, 2015 to 2024", loc="left")
plt.show()
```

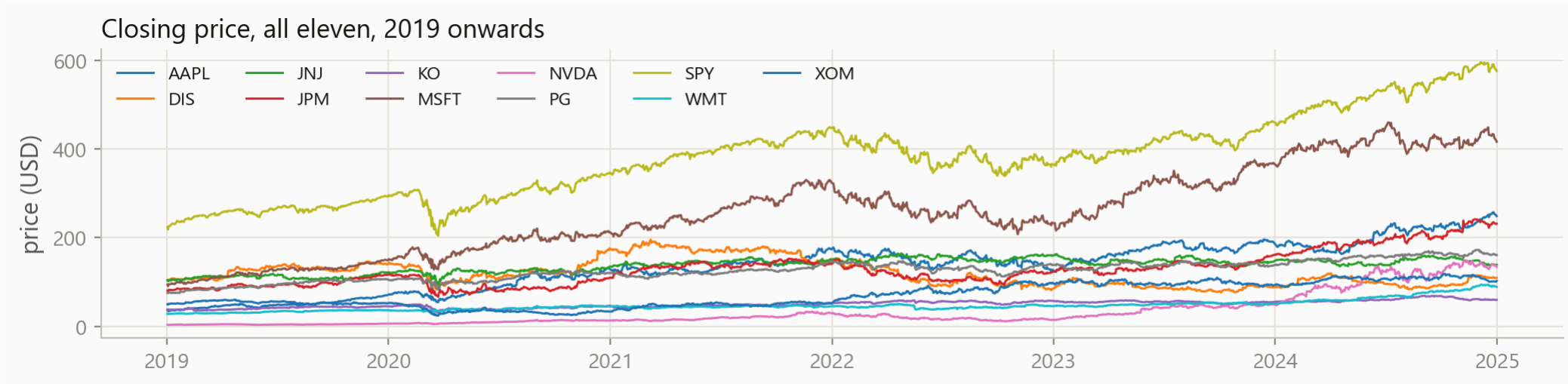
The one-line version

pandas will plot itself, and fills in the axis and the legend for you:

Python Code

```
fig, ax = plt.subplots(figsize=(8, 2.2))
wide.loc["2024", ["AAPL", "KO"]].plot(ax=ax)
plt.show()
```

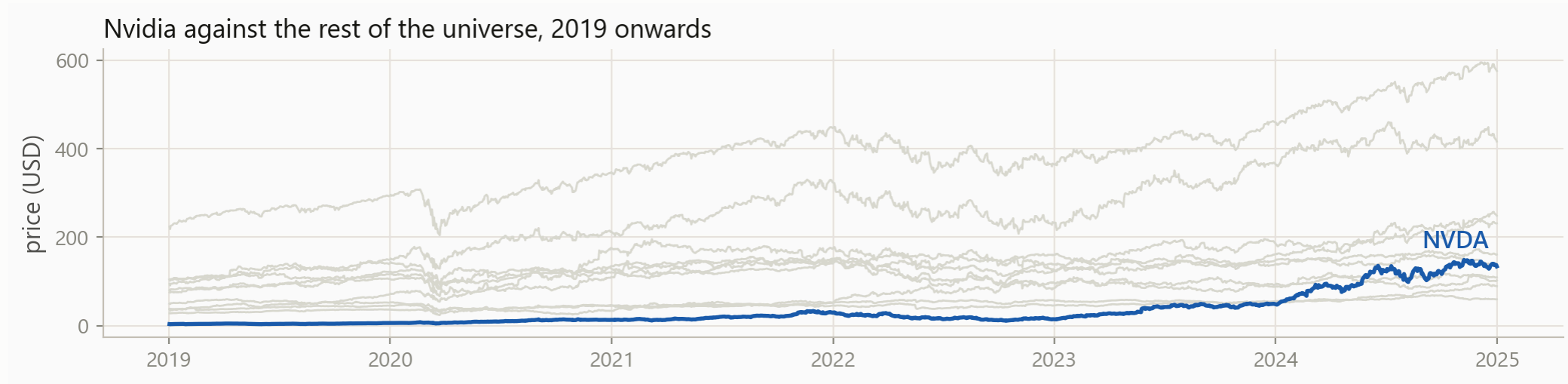
The chart everybody makes first



```
for t in wide.columns:  
    ax.plot(wide.index, wide[t], label=t)  
ax.legend()
```

Eleven lines, eleven colours, and a legend you have to keep looking back at.

Say one thing instead



```
for t in wide.columns:  
    if t != "NVDA":  
        ax.plot(wide.index, wide[t], color="#d8d7d0", linewidth=1.0)  
ax.plot(wide.index, wide["NVDA"], color="#1c5cab", linewidth=2.0)
```

One line in colour, the rest in grey, and no legend to look back at.

What to check before you show it

Does it have a title?

Say what the reader is looking at.

Can you tell the lines apart?

A legend, or a name written on the line.

Is anything decorative?

If it carries no information, remove it.

Are the axes labelled?

With units. "price (USD)", not "price".

Do the bars start at zero?

Lines need not. Bars must.

Would a stranger get it?

Without you standing there to explain.

It can look much better than this



And here is what that cost

```
fig = plt.figure(figsize=(11, 4))
gs = fig.add_gridspec(2, 1, height_ratios=[3, 1], hspace=0.12)
ax, ax2 = fig.add_subplot(gs[0]), fig.add_subplot(gs[1], sharex=ax)
for a in (ax, ax2):
    for side in ("top", "right"):
        a.spines[side].set_visible(False)
ax.plot(s.index, s.values, color="#1c5cab", linewidth=1.5, zorder=3)
for i, alpha in enumerate(np.linspace(0.16, 0.01, 14)):
    ax.fill_between(s.index, s.values * (i / 14), s.values * ((i + 1) / 14),
                    color="#1c5cab", alpha=alpha, linewidth=0, zorder=1)
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("${x:,.0f}"))
ax.axvspan(peak, trough, color="#b8860b", alpha=0.10, zorder=0)
ax.annotate("peak", xy=(peak, s.loc[peak]), xytext=(-52, 22),
            textcoords="offset points", color="#7a5a08",
            arrowprops=dict(arrowstyle="-", color="#c9a227"))
# ... and about thirty more lines for the drawdown panel, the inset
# histogram, the other annotations and the tick formatting
```

Every one of those lines is a small, ordinary call. There are just a lot of them, and none of them is the analysis.

So learn the four calls first

Get the plot right, then make it pretty, and only if it is worth the time.

When you do want it prettier, go to the **matplotlib gallery** at [matplotlib.org](https://matplotlib.org/gallery). It is a wall of example charts, each with the code that made it. Find the picture closest to what you want, and read how it was done.

That is how everybody learns plotting, including the people who do it for a living.

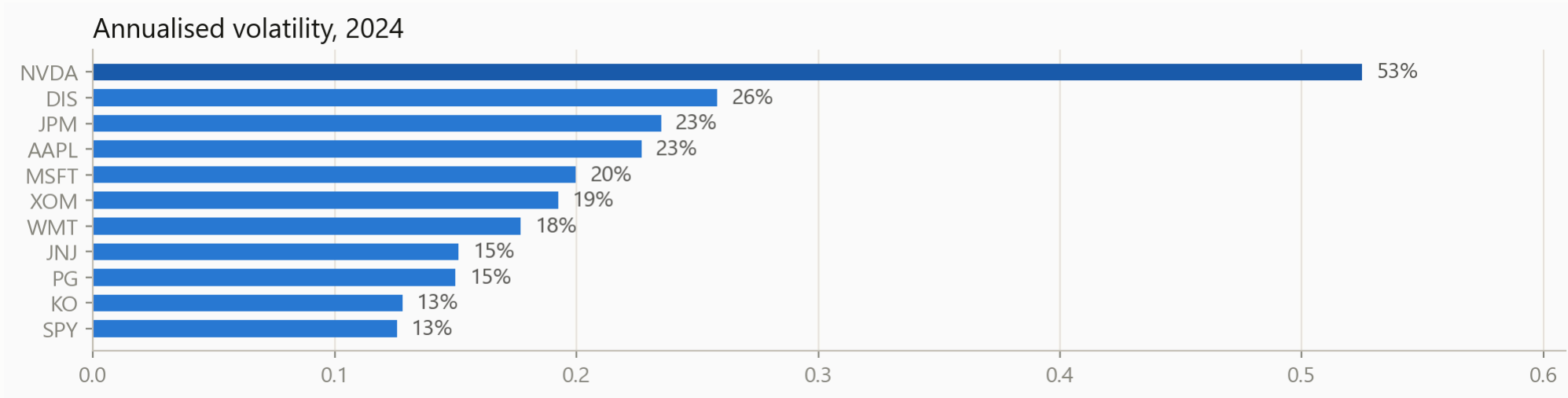
Saving a figure

Now that you have a machine with folders on it:

```
fig.savefig("figures/nvda_2024.png", dpi=200, bbox_inches="tight")
```

`dpi` is the resolution and `bbox_inches="tight"` trims the empty margin. Use `.pdf` instead of `.png` for something that stays sharp at any size, which is what you want in a thesis.

The whole ranking, in four lines



```
wide = prices.pivot(index="date", columns="ticker", values="close")
vol = (wide.loc["2024"].pct_change().std() * np.sqrt(252)).sort_values()

fig, ax = plt.subplots(figsize=(10, 4))
ax.barh(vol.index, vol.values)
```

What stays with you

Your own machine

Install once, `import` in every notebook.

NumPy

Arrays do arithmetic all at once. `@` for matrices.

matplotlib

`fig, ax = plt.subplots()`, then label everything.

Looking things up

Hover, or `help`. It is on your machine, not online.

pandas

`read_csv`, filter, new columns, `groupby`, `pivot`.

Judgment

The form, the scale and the baseline decide what a reader sees.

The course case, Part 3 **CASE**

The risk report grows from two stocks to the whole universe.

Load all eleven with pandas, compute returns and volatility for every one of them at once, and make the figures a reader could act on. Same investigation, no loops.

Case Part 3: this week

`session_03_case.ipynb`, done at home before Session 4:

- load `prices.csv` yourself, and check it before trusting it
- daily returns and annualised volatility for all eleven, with `groupby`
- confirm your Session 2 answer for Apple, computed a different way
- rank the universe by risk, and plot the ranking
- look at how the stocks move together, and what that means for a portfolio

Hints and full solutions are folded into the notebook. Work first, then check.

Before Session 4

HOMEWORK

Exercises

`session_03_exercises.ipynb`: NumPy, pandas and matplotlib, plus practice at looking things up. Hints and solutions folded in.

HOMEWORK

Case Part 3

`session_03_case.ipynb`: the risk report, all eleven stocks, with figures.

DO IT ONCE

Set up locally

Python, VS Code and the three packages. The links are on the course page. Bring any error you get.

Next session: what machine learning actually is. Observations, features and targets, and why a model that fits your data perfectly is usually worthless.